

Android 侧边栏（DrawerLayout）使用详解

Android 侧边栏（DrawerLayout）使用详解

Android 侧边栏核心是系统自带的 `DrawerLayout`（属于 `androidx.drawerlayout` 库），也常结合第三方库简化开发。下面分**原生实现**和**第三方库**两部分讲解：

一、原生 DrawerLayout 基础使用

1. 核心概念

`DrawerLayout` 是一个布局容器，允许将子布局设为**侧边抽屉**（左侧/右侧），主内容区为默认显示区域。

- 侧边栏：通过 `layout_gravity` 标记（`start` / `left` 左侧、`end` / `right` 右侧）
- 主内容：无 `layout_gravity` 的第一个子布局

2. 基本步骤

(1) 添加依赖（AndroidX）

Code block

```
1 dependencies {  
2     // 核心 DrawerLayout  
3     implementation "androidx.drawerlayout:drawerlayout:1.2.0"  
4     // 可选: Material Design 导航组件（搭配 NavigationView）  
5     implementation "com.google.android.material:material:1.12.0"  
6 }
```

(2) 布局文件（res/layout/activity_main.xml）

Code block

```
1 <?xml version="1.0" encoding="utf-8"?>  
2 <androidx.drawerlayout.widget.DrawerLayout  
3     xmlns:android="http://schemas.android.com/apk/res/android"  
4     xmlns:app="http://schemas.android.com/apk/res-auto"  
5     android:id="@+id/drawer_layout"  
6     android:layout_width="match_parent"  
7     android:layout_height="match_parent">
```

```

8
9      <!-- 主内容区域（必须是第一个子布局） -->
10     <LinearLayout
11         android:layout_width="match_parent"
12         android:layout_height="match_parent"
13         android:orientation="vertical">
14
15         <androidx.appcompat.widget.Toolbar
16             android:id="@+id/toolbar"
17             android:layout_width="match_parent"
18             android:layout_height="?attr/actionBarSize"
19             android:background="?attr/colorPrimary"/>
20
21         <TextView
22             android:layout_width="match_parent"
23             android:layout_height="match_parent"
24             android:text="主内容区"
25             android:gravity="center"/>
26     </LinearLayout>
27
28     <!-- 侧边栏（左侧）：NavigationView 简化菜单布局 -->
29     <com.google.android.material.navigation.NavigationView
30         android:id="@+id/nav_view"
31         android:layout_width="wrap_content"
32         android:layout_height="match_parent"
33         android:layout_gravity="start" <!-- start 适配 RTL 布局，优先用 start 而非
left -->
34         app:menu="@menu/nav_menu" <!-- 侧边栏菜单 -->
35         app:headerLayout="@layout/nav_header"/> <!-- 侧边栏头部布局 -->
36
37 </androidx.drawerlayout.widget.DrawerLayout>

```

(3) 配套资源

- 菜单文件（res/menu/nav_menu.xml）：

Code block

```

1 <menu xmlns:android="http://schemas.android.com/apk/res/android">
2     <group android:checkableBehavior="single">
3         <item
4             android:id="@+id/nav_home"
5             android:title="首页"/>
6         <item
7             android:id="@+id/nav_setting"
8             android:title="设置"/>
9     </item>

```

```

10         android:id="@+id/nav_about"
11         android:title="关于"/>
12     </group>
13 </menu>

```

- 头部布局 (res/layout/nav_header.xml) :

Code block

```

1  <LinearLayout
2      xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="match_parent"
4      android:layout_height="160dp"
5      android:padding="16dp"
6      android:orientation="vertical"
7      android:gravity="bottom">
8
9      <ImageView
10         android:layout_width="48dp"
11         android:layout_height="48dp"
12         android:src="@mipmap/ic_launcher"/>
13
14      <TextView
15         android:layout_width="wrap_content"
16         android:layout_height="wrap_content"
17         android:text="用户名"
18         android:textSize="18sp"/>
19 </LinearLayout>

```

(4) Activity 代码

Code block

```

1  import androidx.appcompat.app.AppCompatActivity
2  import android.os.Bundle
3  import androidx.drawerlayout.widget.DrawerLayout
4  import com.google.android.material.navigation.NavigationView
5  import androidx.appcompat.widget.Toolbar
6
7  class MainActivity : AppCompatActivity() {
8      override fun onCreate(savedInstanceState: Bundle?) {
9          super.onCreate(savedInstanceState)
10         setContentView(R.layout.activity_main)
11
12         // 1. 设置 Toolbar 替代 ActionBar (显示侧边栏图标)
13         val toolbar = findViewById<Toolbar>(R.id.toolbar)

```

```
14      setSupportActionBar(toolbar)
15      supportActionBar?.setDisplayHomeAsUpEnabled(true) // 显示返回/菜单图标
16      supportActionBar?.setHomeAsUpIndicator(R.drawable.ic_menu) // 自定义侧边
    栏图标
17
18      // 2. 获取 DrawerLayout 和 NavigationView
19      val drawerLayout = findViewById<DrawerLayout>(R.id.drawer_layout)
20      val navView = findViewById<NavigationView>(R.id.nav_view)
21
22      // 3. 侧边栏菜单点击事件
23      navView.setNavigationItemSelectedListener { menuItem ->
24          when (menuItem.itemId) {
25              R.id.nav_home -> { /* 处理首页点击 */ }
26              R.id.nav_setting -> { /* 处理设置点击 */ }
27              R.id.nav_about -> { /* 处理关于点击 */ }
28          }
29          drawerLayout.closeDrawers() // 点击后关闭侧边栏
30          true
31      }
32
33      // 4. 可选: 通过 Toolbar 图标打开侧边栏
34      toolbar.setNavigationOnClickListener {
35          drawerLayout.openDrawer(androidx.core.view.GravityCompat.START)
36      }
37  }
38 }
```

3. 核心 API

方法	作用
<code>openDrawer(gravity)</code>	打开侧边栏 (GravityCompat.START/END)
<code>closeDrawers()</code>	关闭所有侧边栏
<code>isDrawerOpen(gravity)</code>	判断侧边栏是否打开
<code>addDrawerListener()</code>	监听侧边栏状态 (打开/关闭/滑动)

二、第三方侧边栏库（主流推荐）

原生 `DrawerLayout` 满足基础需求，但第三方库提供更丰富的动画、交互和定制化，以下是常用库：

1. MaterialDrawer（最流行）

- **特点：**Material Design 风格，高度定制（头部、菜单、徽章、动画），支持 Kotlin/Java
- **地址：**<https://github.com/mikepenz/MaterialDrawer>
- **核心优势：**无需写 XML 菜单，纯代码构建，支持多级别菜单、夜间模式、侧边栏叠加等

基本使用（Kotlin）：

Code block

```
1  implementation "com.mikepenz:materialdrawer:9.0.1"
```

Code block

```
1  // 构建侧边栏
2  val drawer = DrawerBuilder()
3      .withActivity(this)
4      .withToolbar(toolbar)
5      .addDrawerItems(
6          PrimaryDrawerItem().withIdentifier(1).withName("首页"),
7          PrimaryDrawerItem().withIdentifier(2).withName("设置"),
8          DividerDrawerItem(), // 分割线
9          SecondaryDrawerItem().withIdentifier(3).withName("关于")
10     )
11     .withOnDrawerItemClickListener { _, position, drawerItem ->
12         // 处理点击
13         true
14     }
15     .build()
```

2. SlidingMenu（经典老牌）

- **特点：**轻量、灵活，支持左右双侧边栏、侧边栏宽度/透明度自定义
- **地址：**<https://github.com/jfeinstein10/SlidingMenu>
- **注意：**维护较少，适合对兼容性要求高的老项目

基本使用：

Code block

```
1  implementation "com.jeremyfeinstein.slidingmenu:library:1.3.0"
```

Code block

```
1  val slidingMenu = SlidingMenu(this)
2  slidingMenu.mode = SlidingMenu.LEFT // 左侧侧边栏
3  slidingMenu.setTouchModeAbove(SlidingMenu.TOUCHMODE_FULLSCREEN) // 全屏滑动触发
4  slidingMenu.setShadowWidthRes(R.dimen.shadow_width)
5  slidingMenu.setMenu(R.layout.sliding_menu) // 侧边栏布局
6  slidingMenu.attachToActivity(this, SlidingMenu.SLIDING_WINDOW)
```

3. Drawer Behavior（增强原生动画）

- **特点：**基于原生 `DrawerLayout`，增强滑动动画（如底部缩放、模糊、渐变）
- **地址：**<https://github.com/matthewrkula/Drawer-Behavior>
- **优势：**兼容原生 API，仅增强动画效果，无侵入性

三、使用建议

1. **优先原生：** `DrawerLayout + NavigationView` 满足 90% 场景，兼容性好、体积小；
2. **复杂需求选第三方：**
 - 需丰富动画/定制化 → `MaterialDrawer`；
 - 需双侧边栏/老项目兼容 → `SlidingMenu`；
 - 仅增强原生动画 → `Drawer Behavior`；
3. **注意适配：**侧边栏宽度建议不超过屏幕 80%，避免遮挡关键内容；
4. **交互规范：**
 - 左侧侧边栏用于导航，右侧用于筛选/设置；
 - 支持滑动（从边缘）、点击 Toolbar 图标打开；
 - 点击菜单/空白区域自动关闭。

四、常见问题

1. **侧边栏遮挡 Toolbar：**确保主内容区包含 Toolbar，且 `DrawerLayout` 是根布局；
2. **RTL 布局适配：**用 `GravityCompat.START/END` 替代 `left/right`；
3. **沉浸式状态栏：**给 `DrawerLayout` 添加 `android:fitsSystemWindows="true"`，并调整侧边栏头部 padding；

4. **菜单选中状态：**调用 `menuItem.setChecked(true)` 标记选中项。

(注：文档部分内容可能由 AI 生成)